

INTERVIEW PREP SERIES

Algorithm

Interview Masterclass

Every Topic. Every Pattern. Every Problem — Decoded.

Dr. Amar Panchal

Founder & Lead Instructor · Career Credentials



Why Algorithms Are the Core of Every Tech Interview

The Reality of FAANG & Product Company Hiring

80%

of FAANG rounds are
algorithm-focused

45

minutes to solve &
explain a problem

3×

higher pass rate with
structured prep

100+

companies use
LeetCode-style tests

What Interviewers Evaluate

- › Problem decomposition ability
- › Data structure selection
- › Time & space complexity analysis
- › Code clarity & edge case handling
- › Communication while coding

Common Candidate Mistakes

- › Jumping to code without thinking
- › Ignoring edge cases entirely
- › Not knowing Big-O trade-offs
- › Memorising solutions vs patterns
- › Freezing under time pressure

The Complete Algorithm Topics Map

Your cheat-sheet — every topic you must master before any interview



Arrays &
Strings



Linked
Lists



Stacks &
Queues



Hash Maps
& Sets



Heaps &
Priority Q



Trees &
BST



Graphs
BFS/DFS



Sorting &
Searching



Dynamic
Programming



Greedy &
Backtrack

Topic 1 & 2 — Arrays, Strings & Linked Lists

The Foundation — Asked in Every Single Interview

ARRAYS & STRINGS

Two Pointers

Pair sum, container with most water, trapping rain

Sliding Window

Max subarray, longest substring, min window

Prefix Sum

Range queries, subarray sum equals k

Sorting Tricks

Sort + search, Dutch flag, interval merge

String Manipulation

Anagrams, palindromes, pattern matching (KMP)

LINKED LISTS

Fast & Slow Pointers

Cycle detection, middle node, palindrome check

Reversal

Reverse list, reverse k-group, rotate list

Merge & Sort

Merge two sorted, sort list $O(n \log n)$

Dummy Node Trick

Remove nth from end, partition list

LRU Cache

HashMap + DLL — premium system design combo

Topic 3 & 4 — Stacks, Queues & Hash Maps / Sets

O(1) Power Tools — Master These For Instant Edge

STACKS

Monotonic Stack

Next greater/smaller element, histogram area

Valid Parentheses

Bracket matching, expression evaluation

Min Stack

$O(1)$ min retrieval with auxiliary stack

Decode String

Nested structure unwinding pattern

QUEUES / DEQUE

BFS Pattern

Level-order traversal, shortest path (unweighted)

Sliding Window Max

Monotone deque for $O(n)$ max queries

Task Scheduler

Greedy + max-heap or queue simulation

Design Circular Q

$O(1)$ enqueue & dequeue with fixed array

HASH MAPS & SETS

Frequency Count

Anagram, top-k frequent, group anagrams

Two Sum Family

Two sum, four sum, subarray sum variants

Prefix Hash

Contiguous subarray with sum/condition

Union-Find

Disjoint sets — component connectivity $O(\alpha)$

Topic 5 & 6 — Trees, BSTs & Graphs

The Most Commonly Tested Topics at Senior Levels



TREES & BST

DFS Traversals

In/Pre/Post order iterative & recursive. Know all 3!

BFS / Level Order

Queue-based. Return by level, zigzag, right side view

Binary Search Tree

Insert/delete/search $O(h)$. Validate BST, kth smallest

Lowest Common Ancestor

LCA — recursive, parent pointer & Euler tour approaches

Tree DP

Max path sum, diameter, house robber III, camera placement

Trie (Prefix Tree)

Insert/search/prefix. Word search, auto-complete, XOR max



GRAPHS

BFS / DFS

Connectivity, path finding, flood fill, bipartite check

Topological Sort

Kahn's (BFS) & DFS. Course schedule, build order

Shortest Path

Dijkstra (weighted), Bellman-Ford (negative edges)

Union-Find

MST via Kruskal, redundant connection, accounts merge

Cycle Detection

DFS coloring (directed), Union-Find (undirected)

Grid Graphs

Islands, walls, rotting oranges, word search

Topic 7 & 8 — Sorting, Binary Search & Heaps

Speed & Selection — The $O(\log n)$ Toolkit



SORTING ALGORITHMS — Know Time, Space, Stability

Algorithm	Best/Avg	Worst	Space	Stable?	Use Case
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Stable	Linked list sort, external sort
Quick Sort	$O(n \log n)$	$O(n^2)$ worst	$O(\log n)$	✗ Unstable	Default in most languages
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ Unstable	In-place guaranteed $O(n \log n)$
Counting/Rad	$O(n+k)$	$O(n+k)$	$O(n+k)$	✓ Stable	Small range integers only

BINARY SEARCH PATTERNS

Classic BS Sorted array, find target — know lo/hi/mid off-by-one

Search in Rotated Modified mid check — pivot detection pattern

BS on Answer Space Min/Max feasibility — Koko eating bananas, ship weight

First/Last Occurrence Left/right boundary binary search template

HEAPS & PRIORITY QUEUES

Top-K Pattern Kth largest, top-k frequent — min-heap of size k

Two Heaps Median of stream — max-heap left + min-heap right

Merge K Sorted Min-heap with (val, list, idx) tuple extraction

Task / Meeting Rooms Greedy scheduling with heap-based conflict checks

Topic 9 & 10 — Dynamic Programming & Greedy Algorithms

The Hard-Hitters — Separate Good from Great Candidates



DYNAMIC PROGRAMMING

0/1 Knapsack

Subset sum, target sum, partition equal subset

Unbounded Knapsack

Coin change, rod cutting, ribbon cutting

Fibonacci / 1D DP

Climbing stairs, house robber, decode ways

LCS / Edit Distance

Longest common subseq, edit distance, SCS

LIS

Longest increasing subseq — $O(n \log n)$ patience sort

Grid DP

Unique paths, min path sum, dungeon game

Interval DP

Burst balloons, palindrome partition, MCM



GREEDY & BACKTRACKING

Classic Greedy

Activity selection, interval scheduling, Huffman

Jump Game

Can reach end? Min jumps? Greedy coverage

Gas Station

Circular route feasibility — cumulative sum trick

Stock Prices

Best time to buy/sell — all DP & greedy variants

Backtracking

Permutations, combinations, subsets — prune early

N-Queens / Sudoku

Constraint propagation + backtrack framework

Complexity Cheat Sheet — Know This Cold

Every interviewer expects you to state time & space complexity without hesitation

$O(1)$	HashMap get/put, array access, stack push/pop
$O(\log n)$	Binary search, balanced BST ops, heap push/pop
$O(n)$	Linear scan, two pointers, BFS/DFS on n nodes
$O(n \log n)$	Merge sort, heap sort, sorting-based solutions
$O(n^2)$	Nested loops, bubble/insertion sort, naive substring
$O(2^n)$	Subsets/power set, Fibonacci naive recursion
$O(n!)$	Permutations, travelling salesman brute force

Pro tip: State complexity before writing code — shows you think before you code.

Dr. Amar's 7-Step Interview Problem Framework

Apply This to Every Single Problem You Encounter

1

Understand & Clarify

Repeat the problem. Ask about input size, edge cases, sorted?, duplicates?, constraints.

2

Work Through Examples

Trace 2-3 examples manually. Find the pattern. Don't skip this — it reveals the algorithm.

3

State the Brute Force

Always start with naive. State complexity. Show you understand the baseline before optimising.

4

Optimise — Spot Pattern

Which topic does this belong to? Two pointers? DP? Graph? Map the problem to a known pattern.

5

State Complexity First

Before writing: 'This will be $O(n)$ time, $O(1)$ space.' Interviewers reward upfront analysis.

6

Code Cleanly

Descriptive variables, modular helper functions, no magic numbers. Talk while you type.

7

Test & Edge Cases

Empty array, single element, negatives, overflow. Dry-run your code line by line out loud.

Topic Priority Matrix & 60-Day Study Plan

Study Smart — Not Everything is Equally Important

PRIORITY MATRIX

MUST KNOW	Arrays, Strings, Two Pointers, Sliding Window, Hash Maps, BFS/DFS, Binary Search, DP 1D	95%	freq
HIGH	Trees, Linked Lists, Stacks, DP 2D, Backtracking, Sorting, Heaps, Graphs, Union-Find	80%	freq
MEDIUM	Tries, Segment Trees, Bit Manipulation, Interval DP, Monotone Stack, Topological Sort	55%	freq
GOOD TO HAVE	AVL/Red-Black Trees, Fenwick Tree, Suffix Array, Network Flow, NP-hard awareness	20%	freq

60-DAY ROADMAP

Days 1–10

Arrays, Strings, Hashing

2-pointer, sliding window, prefix sum, frequency maps. Solve 25–30 problems.

Days 11–22

Lists, Stacks, Queues, Trees

Traversals, LCA, BST, monotone stack. Solve 30–35 problems.

Days 23–35

Graphs, Sorting, Binary Search

BFS/DFS, Dijkstra, topo sort, heap patterns. Solve 30 problems.

Days 36–50

DP Patterns + Greedy

All 8 DP patterns, backtracking. Solve 25–30 hard problems.

Days 51–60

Mock Interviews + Review

Timed mocks, revisit weak topics, system design basics.

Must-Solve 30 Problems Before Any Interview

Curated by Dr. Amar Panchal — Covers All Key Patterns

#	Problem	Pattern	Difficulty	LC
1	Two Sum	Hash Map	Easy	LC 1
2	Best Time to Buy & Sell	Array/Greedy	Easy	LC 121
3	Longest Subst. No Repeat	Sliding Window	Medium	LC 3
4	Container With Most Water	Two Pointers	Medium	LC 11
5	Merge Intervals	Sorting + Merge	Medium	LC 56
6	Product of Array Except Self	Prefix/Suffix	Medium	LC 238
7	Reverse Linked List	Linked List	Easy	LC 206
8	Detect Cycle in LL	Fast/Slow Ptr	Medium	LC 142
9	Valid Parentheses	Stack	Easy	LC 20
10	Min Stack	Stack	Medium	LC 155
11	Binary Tree Level Order	BFS	Medium	LC 102
12	Lowest Common Ancestor	Tree DFS	Medium	LC 236
13	Validate BST	BST	Medium	LC 98
14	Course Schedule	Topological Sort	Medium	LC 207
15	Number of Islands	DFS/BFS Grid	Medium	LC 200
16	Binary Search	Binary Search	Easy	LC 704
17	Search Rotated Array	Modified BS	Medium	LC 33
18	Kth Largest in Stream	Heap	Easy	LC 703
19	Top K Frequent Elements	Heap / Bucket	Medium	LC 347
20	Climbing Stairs	DP Fibonacci	Easy	LC 70
21	House Robber	DP 1D	Medium	LC 198
22	Coin Change	DP Unbounded	Medium	LC 322
23	Longest Inc. Subseq.	DP LIS	Medium	LC 300
24	Longest Common Subseq.	DP LCS	Medium	LC 1143
25	Word Break	DP + Trie	Medium	LC 139
26	Partition Equal Subset	0/1 Knapsack	Medium	LC 416
27	Permutations	Backtracking	Medium	LC 46
28	Combination Sum	Backtracking	Medium	LC 39
29	Jump Game	Greedy	Medium	LC 55
30	Trapping Rain Water	Two Ptr / Stack	Hard	LC 42

Pattern Recognition — How to Identify the Algorithm in 60 Seconds

Read the clue words → Match the pattern → Win the interview

IF: "sorted array" + find target

→ **Binary Search**

IF: "max/min subarray" of size k

→ **Sliding Window**

IF: "two numbers sum to X"

→ **Two Pointers or HashMap**

IF: "shortest path" unweighted graph

→ **BFS**

IF: "shortest path" weighted graph

→ **Dijkstra (min-heap)**

IF: "all combinations / subsets"

→ **Backtracking**

IF: "top K" or "Kth largest/smallest"

→ **Heap (priority queue)**

IF: "overlapping sub-problems + optimum"

→ **Dynamic Programming**

IF: "parentheses / matching brackets"

→ **Stack**

IF: "connected components / groups"

→ **Union-Find or DFS/BFS**

IF: "scheduling / dependencies / order"

→ **Topological Sort**

IF: "word prefix / autocomplete"

→ **Trie (Prefix Tree)**

You Now Have the

Algorithm Interview Blueprint

- ✓ 10 complete topic areas mapped with sub-patterns
- ✓ 30 must-solve problems with pattern labels
- ✓ Big-O cheat sheet to recite in any interview
- ✓ 7-step framework to crack any unseen problem
- ✓ 60-day structured study roadmap

Enroll – Career Credentials



www.careercredentials.in | [@DrAmarPanchal](https://twitter.com/DrAmarPanchal)